

國立台北科技大學

NATIONAL TAIPEI UNIVERSITY OF TECHNOLOGY

電機系實習報告

課程名稱：微處理機實習

作業名稱：LAB08

組別：第 13 組

姓名及學號：

張博翔 b812111001

孫皓 110350450

中 華 民 國 114 年 11 月 10 日

微處理機實習 LAB_08

****基本題 1 (30%) :** 參考範例 [ch4_7_2](#) ,

新增一按鈕 PB0 (可參考範例 [2-4](#)) , 當 PB0 按下時即進行以下的跑馬燈顯示(顯示完即停止):

空白 空白 空白 空白 (註: 空白表示無顯示, 七節均不亮)

空白	空白	空白	0
空白	空白	0	1
空白	0	1	2
0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6
4	5	6	7
5	6	7	8
6	7	8	9
7	8	9	空白
8	9	空白	空白
9	空白	空白	空白
空白	空白	空白	空白

實驗拍照: (拍攝下列的顯示各 1 張 , 共 [8](#) 張)

空白	空白	空白	0
空白	空白	0	1
空白	0	1	2
0	1	2	3
6	7	8	9
7	8	9	空白
8	9	空白	空白
9	空白	空白	空白

電路設計：

P11A-1 ~ P11A-8 接到 6、7、8、9、10、11、12、13 腳位

P11B-1 ~ P11B-4 接到 2、3、4、5 腳位

程式碼：

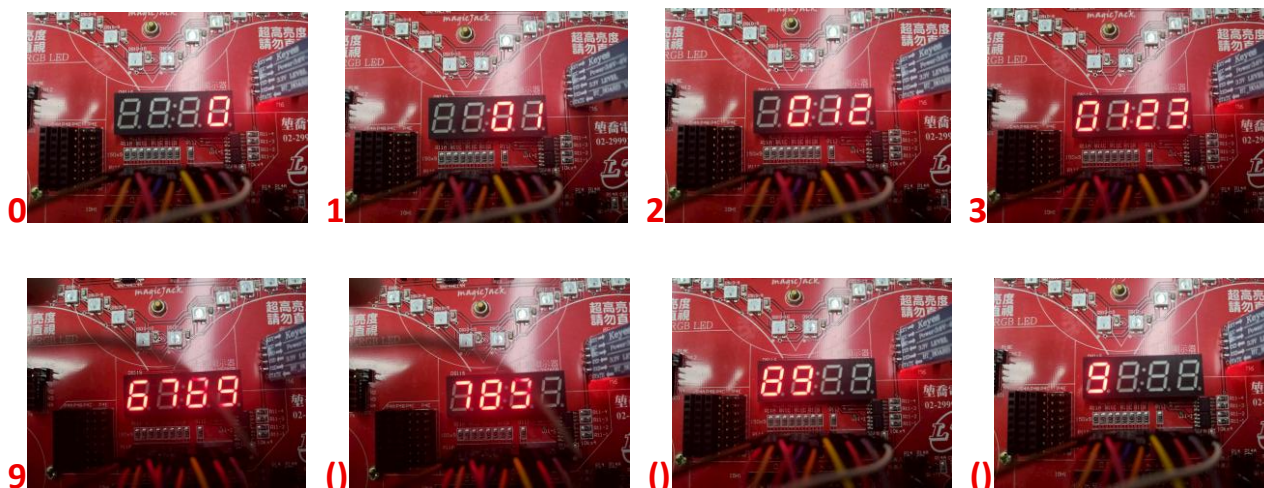
```
/*
範例 ch4_7_2
七節顯示器之 DP、G~A 連接在 13~6 腳
七節顯示器之 com1~com4 連接在 5~2 腳
*/
const int SEG[8] = {6,7,8,9,10,11,12,13}; // 宣告顯示信號接腳
const int com[4] = {2,3,4,5};           // 宣告掃描信號接腳
const int Data[17] = {10,10,10,10,0,1,2,3,4,5,6,7,8,9,10,10,10}; // 宣告顯示內容陣列
//const int Data[11] = {2,0,1,5,0,1,2,3,2,0,1}; // 宣告顯示內容陣列
const int SEG_code[11] = {
                                // 宣告七節顯示碼陣列
    0xc0,0xf9,0xa4,0xb0,0x99,    // 0~4
    0x92,0x82,0xf8,0x80,0x90,
    0xff,} ; // 5~9
byte disp, x=0;                // 宣告 byte 變數
// 初始設定
void setup() {
    for(int i=0;i<8;i++)        // 設定顯示信號接腳為輸出接腳
        pinMode(SEG[i], OUTPUT);
    for(int i=0;i<4;i++)        // 設定顯示掃描接腳為輸出接腳
        pinMode(com[i], OUTPUT);
}
// 主程式
void loop() {
    for(int i=0;i<75;i++) {      // 重複掃描 75 次
        for(int j=0;j<4;j++) {   // 逐位掃描
            for(int k=0;k<4;k++) digitalWrite(com[k],1);
            // 關閉掃描信號(防殘影)
            disp=SEG_code[Data[x+j]]; // 讀取顯示信號
```

```

    for(int k=0;k<8;k++) {      // 輸出顯示信號(並列資料串列輸出)
        if(bitRead(disg,k)) digitalWrite(SEG[k],1);
        else digitalWrite(SEG[k],0);
    }
    digitalWrite(com[3-j],0); // 輸出掃描信號
    delay(1);      // 延遲 1ms(每位數持續顯示 1ms，四位數顯示 4ms)
}
}
if (++x==14) x=0;
}

```

實驗結果：



****基本題 2 (30%):** 參考範例 [ch4_7_3](#)，

在本單元裡，將 4x4 鍵盤與七節顯示器結合，做為簡單的介面。除保持原範例之功能外，新增一功能如下：將 **C 鍵** 做為歸零鍵，按 C 鍵後，則將四位數的七節顯示器全部歸零(均顯示 0)

實驗拍照: (共 5 張)

按 1 鍵(七節顯示器應顯示 0 0 0 1)，拍四位數的七節顯示器 1 張

接著按 3 鍵(七節顯示器應顯示 0 0 1 3)，拍四位數的七節顯示器 1 張

接著按 5 鍵(七節顯示器應顯示 0 1 3 5)，拍四位數的七節顯示器 1 張

接著按 7 鍵(七節顯示器應顯示 1 3 5 7) , 拍四位數的七節顯示器 1 張

接著按 C 鍵(七節顯示器應顯示 0 0 0 0) , 拍四位數的七節顯示器 1 張

電路設計：

P11A-1 ~ P11A-7 接到 6、7、8、9、10、11、12 腳位

P14-2 接到 13 腳位

P11B-1 ~ P11B-5 接到 P4A-5 ~ P4A-1

P4B-5 ~ P4B-2 接到 2、3、4、5 腳位

P4C-5 接到 A1 腳位

P4C-1 ~ P4C-4 接到 P6-1 ~ P6-4

P6-5 ~ P6-8 接到 A5、A4、A3、A2 腳位

程式碼：

```
/*
範例 ch4_7_3_改版
七節顯示器之 G~A 連接在 12~6 腳
七節顯示器之 com1~com4 連接在 5~2 腳
功能新增：
- C 鍵（鍵值 = 11）為歸零鍵，按下後七節顯示器全部歸零
*/
const int SEG[8] = {6,7,8,9,10,11,12}; // 顯示信號接腳
const int scan[4] = {2,3,4,5}; // 掃描信號接腳(row)
const int SEG_code[10] = { // 七節顯示碼陣列
    0xc0,0xf9,0xa4,0xb0,0x99, // 0~4
    0x92,0x82,0xf8,0x80,0x90 }; // 5~9
const int column[4]={A2,A3,A4,A5}; // keyin port
const int BZ=13; // 蜂鳴器
unsigned char KN; // 按鍵值
int disp[4]={0,0,0,0}; // 顯示緩衝區
int DISP; // 顯示變數
```

```

void beep(int pin, int counts); // 嗶聲函數宣告

//=== 初始設定 =====
void setup() {
    for(int i=0;i<7;i++) // 設定顯示信號接腳為輸出
        pinMode(SEG[i], OUTPUT);
    for(int i=0;i<4;i++) {
        pinMode(scan[i], OUTPUT); // 設定掃描接腳為輸出
        pinMode(column[i], INPUT); // 設定行接腳為輸入
    }
    pinMode(BZ, OUTPUT); // 蜂鳴器輸出
}

//=== 主程式 =====
void loop() {
    for(int i=0;i<4;i++) { // 掃描週期
        // 關閉七節顯示器(防殘影)
        for(int j=0;j<8;j++)
            digitalWrite(SEG[j],1);
        // 輸出掃描線
        for(int j=0;j<4;j++) digitalWrite(scan[j],1);
        digitalWrite(scan[i],0); // 選擇第 i 位
        // 輸出七節顯示器顯示信號
        DISP=SEG_code[disp[i]]; // 讀取顯示碼
        for(int j=0;j<8;j++) {
            if(bitRead(DISP,j)) digitalWrite(SEG[j],1);
            else digitalWrite(SEG[j],0);
        }
        // 判讀按鍵狀態
        for(int col=0;col<4;col++) {
            if(!digitalRead(column[col])) { // 有按鍵
                while(!digitalRead(column[col])); // 等待放開
                KN=4*i+col; // 計算鍵值
                beep(BZ,1); // 嗶一聲

                //==== 按鍵功能判斷 ====
                if (KN>=0 && KN<10) { // 若為數字鍵
                    for(int j=0;j<3;j++)
                        disp[3-j]=disp[2-j]; // 移位
                }
            }
        }
    }
}

```

```

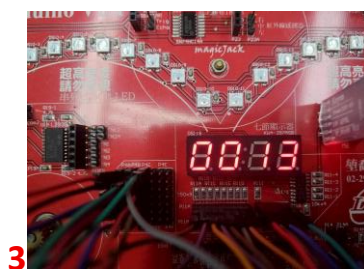
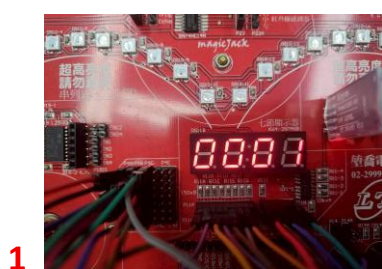
        disp[0]=KN;    // 鍵值放入個位數
    }
    else if (KN == 12) {    // 若為 C 鍵 (鍵值 11)
        for (int j=0; j<4; j++)
            disp[j] = 0;    // 全部歸零
        //beep(BZ,2);    // 雙響提示(可略)
    }

    delay(3);    // 延遲 3ms
    break;    // 跳出內層迴圈
}
}
}
}

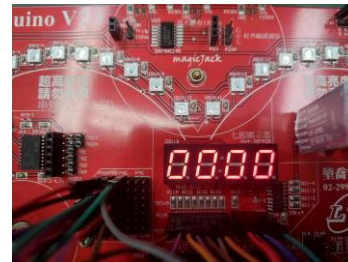
//=== 響聲函數 =====
void beep(int pin, int counts) {
    for(int i=0;i<counts;i++) {    // 執行 counts 次
        tone(pin,1000,100);    // 發聲(1kHz,0.1 秒)
        delay(100);    // 靜音(0.1 秒)
    }
}
}

```

實驗結果：



clear



****基本題 3 (40%)：**參考範例 [ch4_8_1](#)，

本單元所設計的計時器，計時範圍為 00:00 到 59:59，請試著改為 0000 到 9999，每隔 0.5 秒加 1。當顯示 10 或 10 的倍數時，嗶一聲。

實驗結果：請附錄影檔(檔名為“**基本題 3 錄影**”)記錄七節數字顯示從 0000 到 0020 的變化過程。

電路設計：

P11A-1 ~ P11A-7 接到 6、7、8、9、10、11、12 腳位

P14-2 接到 13 腳位

P11B-1 ~ P11B-4 接到 2、3、4、5 腳位

P11B-5 接到 A1 腳位

程式碼：

```
/*
  基本題 3 - 計時器 0000-9999
  七節顯示器之 G~A 連接在 12~6 腳
  七節顯示器之 com1~com4 連接在 5~2 腳
  蜂鳴器 BZ 連接在 13 腳
  每 0.5 秒加 1，顯示 10 或 10 的倍數時嗶一聲
*/
const int SEG[7] = {6,7,8,9,10,11,12}; // 宣告顯示信號接腳
const int scan[4] = {2,3,4,5}; // 宣告掃描信號接腳(scan)
const int SEG_code[10] = { // 宣告七節顯示碼陣列
    0xc0,0xf9,0xa4,0xb0,0x99, // 0~4
    0x92,0x82,0xf8,0x80,0x90 }; // 5~9
const int BZ=13,DD=A1; // 蜂鳴器,中間兩點
int disp[4]={0,0,0,0}; // 宣告顯示緩衝區陣列
int DISP; // 宣告顯示變數
unsigned long x0,x1,x2; // 宣告時間變數
```

```

int counter = 0;           // 計數器 0-9999
void beep(int pin, int counts);           // 宣告嗶聲函數
// 初始設定
void setup() {
    for(int i=0;i<7;i++)           // 設定顯示信號接腳為輸出接腳
        pinMode(SEG[i], OUTPUT);
    for(int i=0;i<4;i++)           // 設定顯示掃描接腳為輸出接腳
        pinMode(scan[i], OUTPUT);
    pinMode(BZ, OUTPUT);           // 設定蜂鳴器接腳為輸出接腳
    pinMode(DD,OUTPUT);           // 設定中間兩點接腳為輸出接腳
    digitalWrite(DD,1);           // 關閉中間兩點（不需要顯示）
    x0 = millis();               // 記錄初始時間
}
// 主程式
void loop() {
    for(int i=0;i<4;i++) {         //掃描週期
        //關閉掃描線(防殘影)
        for(int j=0;j<4;j++) digitalWrite(scan[j],1); //掃描信號全部為 1
        //輸出七節顯示器顯示信號
        DISP=SEG_code[disp[i]];    // 讀取顯示信號
        for(int j=0;j<8;j++) {      // 並列資料串列輸出
            if(bitRead(DISP,j)) digitalWrite(SEG[j],1);
            else digitalWrite(SEG[j],0);
        }
        //輸出掃描線
        digitalWrite(scan[i],0);    //輸出掃描線
        delay(1);                   //延遲 1ms
    }
    //時間處理
    x1=millis();                   //查詢時間
    x2=x1-x0;                      //計算時間差
    if(x2>=500) {                  //0.5 秒時間差
        x0=x1;                     //記錄時間
        counter++;                 //計數器加 1

        if(counter > 9999) {       //超過 9999 則歸零
            counter = 0;
        }
    }
}

```

```

// 檢查是否為 10 或 10 的倍數
if(counter > 0 && counter % 10 == 0) {
    beep(BZ,1);      //嗶一聲
}

// 將 counter 轉換為四位數顯示
disp[0] = counter % 10;      //個位數
disp[1] = (counter / 10) % 10;    //十位數
disp[2] = (counter / 100) % 10;   //百位數
disp[3] = (counter / 1000) % 10;  //千位數
}
}
//=== 嗶聲函數 =====
void beep(int pin, int counts) {
    for(int i=0;i<counts;i++) {    //執行 counts 次
        tone(pin,1000,100);      //發聲(1kHz,0.1 秒)
        delay(100);              //靜音(0.1 秒)
    }
}
}

```

實驗結果：

參考連結：

https://drive.google.com/drive/folders/1ULM1YXwEnMpiK_quZTFSjmAfJoNLqV09?usp=sharing

心得：

張博翔：

第一題七節顯示器是以 4 個資料讀取，並逐漸移位達到跑馬燈效果，運用 SEG_CODE 內資料的位置編排建立 Data 陣列，並且以四位數逐步平移掃描，作為位置顯示 SEG_CODE 陣列內的數值，而因為到 Data 第 14 位後已經沒 4 位數掃描，因此結尾以 14 為界線，當 Data 獨到第 14 個結束後，下一個會回到 0 開始讀取。

第二題則是在鍵位判斷時加入條件，當小於 10 並大於等於 0 時，資料右移一個(其節顯示器左移)，並且將新輸入的值賦值給第 0 個位置，另一個條件則是當輸入為 12 時用迴圈將所有值歸零並達成 clear。經過這次經驗對七節顯示器與按鍵的應用有更進一步的理解，也讓我們

對期末的報告應用此部件有更進一步的想法。

孫皓：

本次實驗內容涵蓋跑馬燈、鍵盤掃描、顯示器更新與計時器邏輯，整體難度比前幾週高，但也讓我更熟悉實際應用的流程。基本題 1 要在按鍵觸發後播放限定長度的跑馬燈動畫，必須精準掌握顯示陣列的順序。基本題 2 加入 C 鍵歸零功能，讓顯示器能快速回到初始狀態，更像是在做小型電子裝置的介面設計。基本題 3 則把計時器升級為四位數遞增，每經過整十需發出提示音，讓我實際體驗到計時器運作與蜂鳴器控制的搭配。這次實驗讓我更理解人機介面的運作方式，也對陣列與流程控制的概念更加熟悉，覺得很有收穫。